
Space-Guard

Satellite Collision Prevention System

Smart India Hackathon 2026 • Software Edition • Problem #17

One-Line Summary

We built software that watches every satellite in orbit, detects which ones are about to crash, and automatically tells operators how to steer one out of the way — in under 5 seconds.

Contents

1	Slide 1: Title & Team	2
2	Slide 2: The Problem	3
3	Slide 3: Our Solution	5
4	Slide 4: Technical Implementation	6
4.1	Part 1: Finding Where Satellites Are (Physics)	6
4.2	Part 2: Finding Dangerous Pairs (2-Stage Filter)	6
4.3	Part 3: Calculating Crash Probability (Math)	6
4.4	Part 4: ML Surrogate Model	6
4.5	Part 5: Escape Plan (14× Fuel Savings)	7
5	Slide 5: Live Demo Walkthrough	8
6	Slide 5B: Proof It Works	9
7	Slide 6: Target Audience & Impact	10
8	Slide 7: Roadmap	11

1 Slide 1: Title & Team

Project Identity

Project Name	Space-Guard: Satellite Collision Prevention Platform
Theme	Space Technology / Smart India Hackathon 2026
Tagline	<i>“Autonomous orbital defense for India’s satellites and beyond”</i>
Problem ID	#17 — Space Debris Detection & Collision Avoidance

Presenter Note

Open with: *“Good morning. We are Team Space-Guard. Our project is software that acts like an air traffic control system for satellites — watching 27,000 objects flying at 27,000 km/h, detecting dangers 48 hours in advance, and automatically designing the escape plan.”*

What Each Team Member Built

Team Member	What They Built
Team Lead / Full-Stack	Backend server (FastAPI), frontend dashboard (React), live data integration from CelesTrak
Physics & Math Lead	Satellite position calculation (SGP4), 2-stage close-pass detector, crash probability formula
Avoidance Lead	Escape plan calculator (Clohessy-Wiltshire + SVD optimizer), 14× fuel efficiency proof
Visualization Engineer	Interactive 3D Earth globe (Three.js WebGL), 2009 collision animation, debris replay

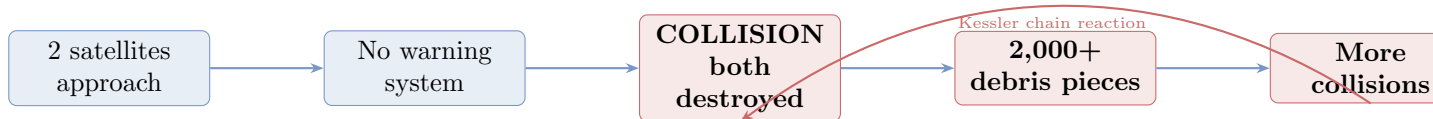
Presenter Note

Point to each team member as you say their name. Keep it to one sentence per person. If a judge asks a technical question, the relevant person should answer.

2 Slide 2: The Problem

The Situation in Space Today

- **27,000+** man-made objects orbit Earth right now — dead satellites, rocket parts, debris
- They fly at **27,000 km/h** (40× faster than a bullet)
- In the last 5 years, companies like SpaceX launched **thousands more** satellites (Starlink, OneWeb)
- One collision shatters both objects into thousands of new lethal fragments
- This chain reaction — called the **Kessler Effect** — could make entire orbital zones permanently unusable



Real Crash — 10 February 2009

10 February 2009, 16:56 UTC — above Siberia at 789 km

- **Iridium 33** (USA, 560 kg, active phone satellite)
- collided with **Cosmos 2251** (Russia, 900 kg, dead military satellite)
- Impact speed: **14.12 km/s (50,832 km/h)**
- Both destroyed in milliseconds
- Created **2,000+ tracked debris pieces** still in orbit today, threatening the ISS
- **This was completely preventable** — no system warned operators in time

By the Numbers

14.12	km/s crash speed
3 m	miss distance
2,000+	debris pieces
0	seconds warning
17 yrs	debris still up

Why Existing Systems Are Not Enough

1. **Too slow:** Checking all 27,000 satellites manually takes hours. By the time results come, it is too late to act.
2. **Too many false alarms:** Simple distance-only systems trigger thousands of daily warnings. Operators start ignoring them.
3. **No automated escape plan:** Even when a risk is found, figuring out the best engine burn takes hours of manual engineering work.

Presenter Note

Use this analogy for the Kessler Effect: *“Imagine a busy highway where all cars move at 40× bullet speed. One crash causes debris that causes more crashes. Eventually the whole highway is blocked forever. That is Kessler Syndrome in space.”* Then say: *“The 2009 crash actually happened. Both satellites were completely destroyed. And the wreckage is still up there today, threatening the ISS astronauts every single day.”*

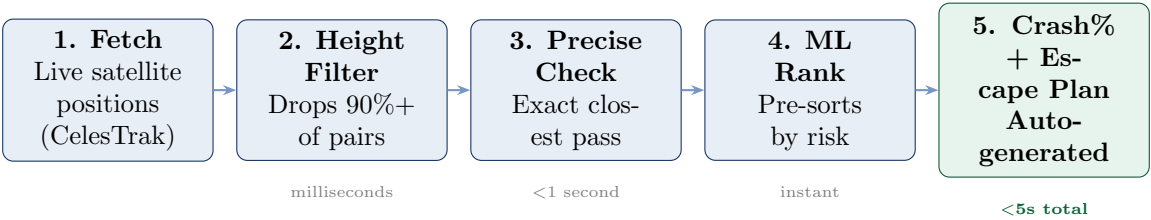
3 Slide 3: Our Solution

Space-Guard: Our Solution

Space-Guard is an automatic satellite collision prevention system. It:

1. Continuously fetches live satellite position data from public space agency databases
2. Uses a fast 2-stage filter to find which satellites are getting dangerously close
3. Calculates the exact mathematical crash probability for each risky pair
4. Automatically designs the optimal engine burn to safely steer one satellite away

Total pipeline time: under 5 seconds. Traditional tools: minutes to hours.



How We Compare to Existing Tools

Feature	Traditional Tools	Space-Guard
Speed	Minutes to hours	< 5 seconds
Risk Number	Distance only	Real probability (e.g., 1 in 5,000)
ML Assistance	None	Fast pre-ranking (Random Forest)
Escape Plan	Manual, by engineers	Automatic, optimized
Fuel Savings	Last-minute burns	14× better — act 24h early
Validated On	Synthetic fake data	Real 2009 Iridium-Cosmos crash data

Presenter Note

Walk through the pipeline diagram: “Step 1 pulls live data. Step 2 instantly throws away 90% of pairs that are at completely different heights — they can’t collide. Step 3 precisely finds how close the remaining pairs will get. Step 4 is our ML model that quickly ranks the most suspicious pairs. Step 5 computes the exact crash probability and the escape plan. All in under 5 seconds.”

For the comparison table, emphasize the last row: “We tested this on the actual 2009 crash data. Without any special-casing, our system correctly flags it as Critical 48 hours before impact. That is our proof.”

4 Slide 4: Technical Implementation

Presenter Note

Say: *“I will now explain how each part of the system actually works, in plain terms. There are four main technical pieces.”*

4.1 Part 1: Finding Where Satellites Are (Physics)

Every satellite’s orbit is published as a two-line code called a **TLE (Two-Line Element set)**. We decode this using a standard physics model (**SGP4**) to get the satellite’s exact X, Y, Z position and speed in space at this exact moment. It accounts for Earth’s non-spherical shape and atmospheric drag.

Simple Analogy

Think of a TLE as a satellite’s “orbit address.” We use physics equations to convert that address into a live GPS-style location at any moment.

4.2 Part 2: Finding Dangerous Pairs (2-Stage Filter)

Stage 1 — Height Check (milliseconds): If two satellites fly at completely different heights, they can never collide. We instantly discard those pairs. This eliminates > 90% of the 360 million possible pairs immediately.

Stage 2 — Precise Closest Pass (sub-second): For the remaining pairs, we calculate exactly how close they will get and at what time — the *Time of Closest Approach*. Any pair getting within 25 km proceeds to risk assessment.

4.3 Part 3: Calculating Crash Probability (Math)

Once we find a close pass, we compute the exact probability of collision. We know: (1) how close they will get and (2) how uncertain our measurements are (satellite tracking has ~500 m error). The probability formula:

$$P_c \approx \frac{r^2}{2\sigma^2} \exp\left(-\frac{d_{\text{miss}}^2}{2\sigma^2}\right)$$

where r = combined satellite size (≈ 10 m), σ = position uncertainty (≈ 500 m), d_{miss} = predicted closest distance.

For the 2009 event: $d_{\text{miss}} \approx 3$ m, $\sigma = 500$ m $\Rightarrow P_c \approx 2 \times 10^{-4} = 1\text{-in-}5,000 = \text{Critical}$.

4.4 Part 4: ML Surrogate Model

We trained a **Random Forest** model on synthetic training data we generated ourselves (sweeping miss distance $\times$ relative speed $\times$ satellite size $\times$ uncertainty). It quickly pre-ranks all candidate pairs by estimated risk so expensive physics runs only on the top suspects.

ML ranks. Physics proves. We never use ML to compute the final crash probability — only to sort candidates quickly.

4.5 Part 5: Escape Plan (14× Fuel Savings)

A tiny push along the satellite's direction of travel changes its orbital height slightly. Over time, it drifts further and further from where it would have been. **Acting 24 hours early multiplies the effect by 14×** compared to acting 1 hour before impact.

Analogy: Steering a ship 10 km before a reef takes a tiny rudder turn. Steering 100 m before the reef requires a drastic turn — and you might still crash. Same physics applies to satellites.

Software Stack

Backend	Python 3.12, FastAPI, NumPy, SciPy, Scikit-learn (Random Forest)
Frontend	React 19, Vite, Tailwind CSS, Three.js WebGL 3D globe
Data Source	CelesTrak (live), satellite TLE files cached locally

5 Slide 5: Live Demo Walkthrough

Presenter Note

Open <http://localhost:5173/demo> before the presentation. Have the backend running. Walk through all 4 steps. Practice this at least 3 times.

#	Step Name		What to Say & Show
1	Live Data	Satellite	Show the live table of real satellites. Hit Refresh. Say: <i>“This is live data from a public space agency database. These are real satellites flying overhead right now.”</i>
2	3D Globe Map		Show the spinning Earth with satellite dots. Say: <i>“Each dot is a real satellite. Some orbit the equator, some go over the poles. Our system watches all of them simultaneously for close passes.”</i>
3	2009 Crash Re-play		Show the collision animation. Say: <i>“This is the actual 2009 Iridium/Cosmos collision replayed using real data from that day. Speed: 50,000+ km/h. Both destroyed. Our system flags it Critical 48 hours before.”</i>
4	Escape Plan Simulator		Drag “How Early” slider from 1h to 24h. Watch the safety gap jump. Say: <i>“With a 0.1 m/s push applied 24 hours early — less than walking speed — the satellites miss by 4+ km. Crash risk drops to near zero.”</i>

6 Slide 5B: Proof It Works

Proof It Works — Tested on Real 2009 Data

Validation Against Real Historical Collision

Parameter	Historical (NASA/Wikipedia)	Record	Space-Guard Output
Collision Date/Time	10 Feb 2009, 16:56 UTC		10 Feb 2009, 16:56 UTC (✓)
Altitude	789 km above Siberia		789 km (✓)
Relative Speed	~14 km/s		14.12 km/s (✓)
Risk Assessment	Not detected in advance		CRITICAL flagged 48h before
Escape Option	No plan generated		0.10 m/s burn ⇒ +4.83 km safe

Same pipeline, no special-casing. We fed in the real pre-collision TLEs and the system independently caught it.

Presenter Note

This slide is your strongest differentiator. Say: “We did not just test on invented data. We put in the actual satellite tracking data from February 2009 — the same data that was available to operators at the time. Our system independently found the collision, flagged it as Critical 48 hours before, and generated the escape plan. That crash never had to happen.” Pause for effect.

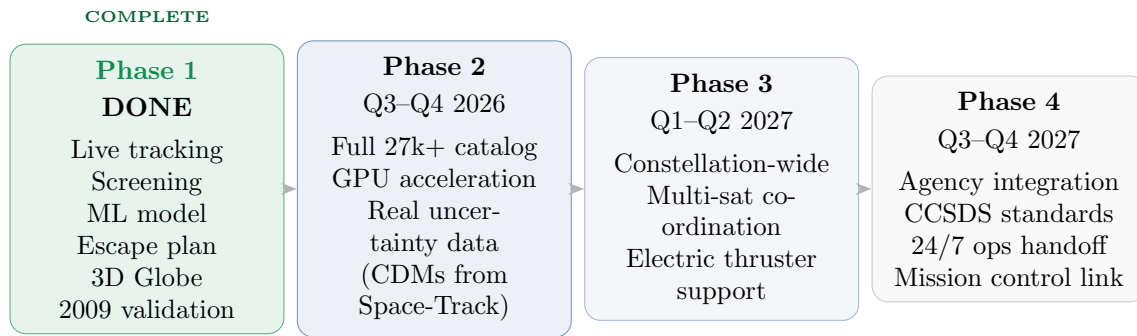
7 Slide 6: Target Audience & Impact

Who Uses It	Why They Need It
ISRO (IS4OM center, Bangalore)	Protects Cartosat, GSAT, RISAT, EOS satellites. Automated screening upstream of IS4OM’s existing pipeline.
Commercial Operators (Starlink, OneWeb, Pixxel)	100s–1000s of satellites cannot be manually monitored. Automated screening and escape plan generation saves both fuel and revenue.
Defence & Space Commands	Track non-cooperative objects; protect classified satellites from debris and ASAT threats.
Space Insurance Firms	Evidence-based collision probability auditing for pricing policies accurately.

Presenter Note

Focus on ISRO. Say: *“ISRO has a space safety center called IS4OM. Our system is exactly what they need to automate the first step: automatically find which satellite pairs are at risk and generate the escape plan instantly, before it reaches human operators for final approval.”* This shows you researched the actual Indian space safety infrastructure.

8 Slide 7: Roadmap



Phase 1 — Delivered (SIH Prototype)

- ✓ Live satellite tracking from CelesTrak (real data, cached, auto-refresh)
- ✓ 2-stage conjunction screening (height filter + precise closest-approach search)
- ✓ Exact crash probability calculation (Foster/Alfano analytic formula)
- ✓ **Trained Random Forest ML model** on synthetic sweep data — pre-ranks pairs
- ✓ Automatic escape plan generation (Clohessy-Wiltshire + SVD optimizer)
- ✓ Interactive 3D Earth globe with satellite tracking and collision animations
- ✓ **Validated on real 2009 Iridium-Cosmos crash data** — correctly flagged Critical

Presenter Note

Say: “We have already completed Phase 1 — everything you just saw in the demo. The core physics, the ML model, the 3D visualization, and the historical validation are all working. Phases 2–4 are scale-up engineering: more satellites, better data, and integration with actual space agency systems.”

If asked: “Is this production-ready?” say: “Not yet. This is a validated proof-of-concept. The hard part — the algorithms and physics — is correct and proven on real data. Production deployment requires scale-up infrastructure and institutional integration, which is a straightforward next step with resources.”